BYU CAPSTONE TEAM 33

Capstone Setup Guide

Step-by-step setup and configuration instructions

PREPARED BY

BYU Capstone Team 33

GENERATED

April 07, 2026

FORMAT

Paged.js PDF

Table of contents

OVERVIEW

Capstone Documentation **7**

Architecture Overview 7

Architecture Diagram 9

Order to Create Architecture 10

BUILD ORDER

Azure AI Foundry **11**

Purpose 11

URL 11

Run Location 11

Before You Start 11

Context 11

Steps 11

Step 1: Create the Azure AI Foundry hub 11

Step 2: Create the project workspace 12

Step 3: Deploy an AI Model 12

Step 4: Locate Endpoints and Keys 13

What You Just Set Up 14

Azure AI Search **15**

Purpose 15

URL 15

Run Location 15

Before You Start 15

Context 15

Steps 15

Step 1: Create the Azure AI Search service 16

Step 2: Locate the endpoint and admin key 16

What You Just Set Up 16

Ubuntu Virtual Machine 17

Purpose 17

Run Location 17

Before You Start 17

Context 17

Steps 17

Step 1: Create the Azure VM 17

Step 2: Connect to the VM 18

Step 3: Install the base packages on the VM 19

Step 4: Copy files to the VM when a later page requires it 19

What You Just Set Up 19

WireGuard 20

Purpose 20

Run Location 20

Before You Start 20

Context 20

Steps 20

Step 1: Install WireGuard on the Ubuntu VM 20

Step 2: Generate the server key pair 21

Step 3: Create the server configuration 21

Step 4: Enable IPv4 forwarding 22

Step 5: Allow WireGuard through the Azure NSG 22

Step 6: Start WireGuard and enable it at boot 23

Step 7: Configure a client (Windows) 24

Step 8: Validate the connection 25

Step 9: Optional full-tunnel mode 26

What You Just Set Up 26

Docker 27

Purpose 27

Run Location 27

Before You Start 27

Context 27

Steps 27

Step 1: Install Docker repository prerequisites 27
Step 2: Add Docker's official apt repository 28
Step 3: Install Docker Engine and Compose 28
Step 4: Enable Docker at boot 28
Step 5: Allow the VM user to run Docker without sudo 28
Step 6: Validate the installation 29
What You Just Set Up 29
LiteLLM 30
Purpose 30
URL 30
Run Location 30
Before You Start 30
Context 30
Steps 30
Step 1: Create the LiteLLM configuration directory 31
Step 2: Create config.yaml 31
Step 3: Run LiteLLM in Docker 32
Step 4: Validate the LiteLLM endpoint 32
Step 5: Update the configuration later if needed 33
What You Just Set Up 33
RAG Website 34
Purpose 34
URL 34
Run Location 34
Before You Start 34
Context 34
Steps 35
Step 1: Download the starter bundle 35
Step 2: Upload the bundle to the Ubuntu VM 35
Step 3: Extract the bundle on the Ubuntu VM 35
Step 4: Edit the configuration files 35
Step 5: Create the Python virtual environment and install dependencies 36
Step 6: Install the systemd unit 36
Step 7: Start the service and enable it at boot 37
Step 8: Validate the website 37

What You Just Set Up 37

MCP Server 38

Purpose 38
URL 38
Run Location 38
Before You Start 38
Context 38
Steps 39
Step 1: Download the starter bundle 39
Step 2: Upload the bundle to the Ubuntu VM 39
Step 3: Extract the bundle on the Ubuntu VM 39
Step 4: Edit the configuration files 39
Step 5: Create the Python virtual environment and install dependencies 40
Step 6: Install the systemd units 40
Step 7: Start the services and enable them at boot 41
Step 8: Validate the services 41
What You Just Set Up 41

OpenWebUI 42

Purpose 42
URL 42
Run Location 42
Before You Start 42
Context 42
Steps 43
Step 1: Start OpenWebUI in Docker 43
Step 2: Create the first admin account 43
Step 3: Create additional user accounts 43
Step 3a: Optional — run without accounts 44
Step 4: Validate the container 44
Step 4: Connect OpenWebUI to the MCP tools 45
Step 5: Reset the instance later if needed 45
What You Just Set Up 45

Visual Studio Code (Cline) 46

Purpose 46

Run Location 46
Before You Start 46
Context 46
Steps 46
Step 1: Install Visual Studio Code 46
Step 2: Install the Cline extension 47
Step 3: Configure Cline to use LiteLLM 47
Step 4: Connect Cline to the MCP server 47
Step 5: Confirm the connections 48
What You Just Set Up 48

REFERENCE

Key Terms 49
Core AI Concepts 49
AI (Artificial Intelligence) 49
LLM (Large Language Model) 49
RAG (Retrieval-Augmented Generation) 49
MCP (Model Context Protocol) 49
MCPO (MCP OpenAPI Proxy) 49
Embedding / Vector Embedding 50
Chunking 50
Token / TPM (Tokens Per Minute) 50
Inference 50
Azure & Cloud Services 50
Azure AI Foundry 50
Azure AI Search 50
Azure Government 51
API (Application Programming Interface) 51
API Key 51
Resource Group 51
VM (Virtual Machine) 51
Deployment 51
Networking & Security 51
VPN (Virtual Private Network) 51
WireGuard 52

VNet (Virtual Network) 52
NSG (Network Security Group) 52
NAT (Network Address Translation) 52
NIC (Network Interface Card) 52
Split Tunnel / Full Tunnel 52
DNS (Domain Name System) 52
SSH (Secure Shell) 53
Infrastructure & Application Stack 53
Docker / Container 53
Docker Compose 53
systemd 53
Flask 53
OpenAPI 53
LiteLLM 54
OpenWebUI 54
Cline 54
Web Browser **55**
Purpose 55
URLs 55
Azure API **56**
Purpose 56

Capstone Documentation

If you are building the stack from scratch, follow the pages in the **Build Order** section of the left navigation.

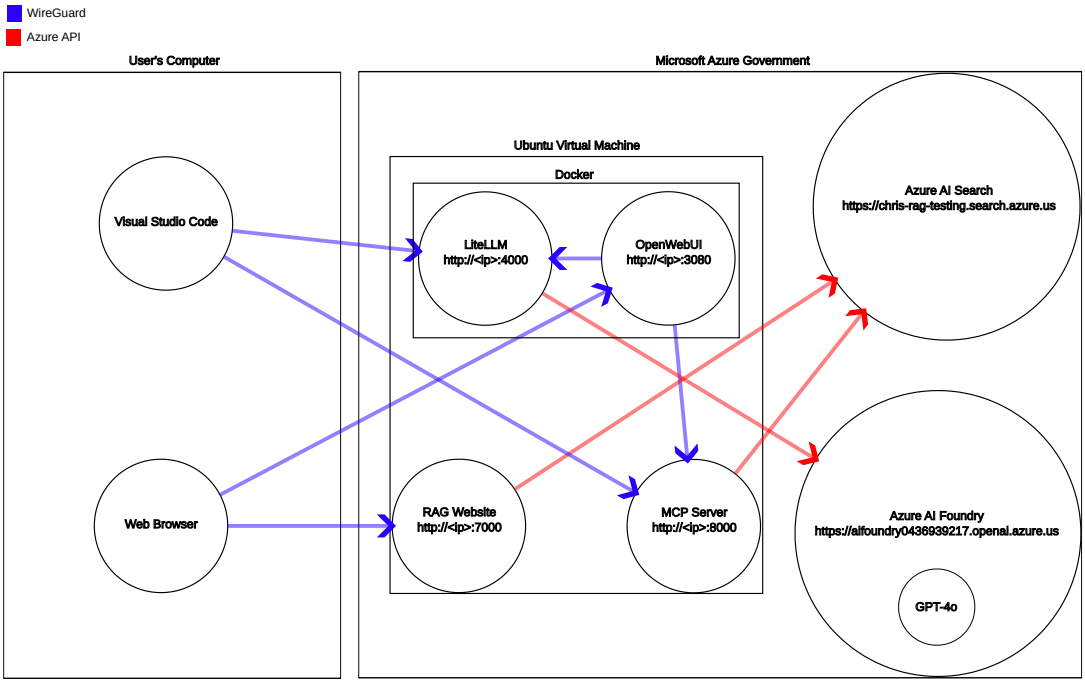
Architecture Overview

In plain language, the architecture works like this:

1. **Azure AI Foundry** hosts the deployed models and provides the API keys and endpoints the VM-based services use.
2. **Azure AI Search** stores the searchable chunks that power retrieval.
3. The **Ubuntu VM** is the private application host for LiteLLM, OpenWebUI, the RAG website, and the MCP server.
4. **WireGuard** gives approved users private access to the VM over `10.55.55.1` instead of exposing those apps publicly.
5. **LiteLLM** gives clients a single OpenAI-compatible endpoint for the deployed Azure model.
6. The **RAG website** creates and manages indexes in Azure AI Search.
7. The **MCP server** exposes those search indexes as callable tools.
8. **OpenWebUI** and **Cline** are the main user interfaces that connect to LiteLLM and optionally the MCP tools.

Capstone Documentation

Architecture Diagram



Order to Create Architecture

1. Azure AI Foundry
2. Azure AI Search
3. Ubuntu Virtual Machine
4. WireGuard
5. Docker
6. LiteLLM
7. RAG Website
8. MCP Server
9. OpenWebUI
10. Visual Studio Code (Cline)

BUILD ORDER

Azure AI Foundry

Purpose

Create the Azure AI Foundry resources and model deployment used later by LiteLLM and the RAG website.

URL

Project endpoint: `https://<azure-openai-endpoint>.openai.azure.us/`

Run Location

Azure portal.

Before You Start

- An Azure Government subscription with permission to create AI resources
- A resource group in the target region
- Available model quota in the region where the deployment will run

---

Context

This page creates three things: an Azure AI Foundry hub, a project attached to that hub, and at least one deployed model. Later pages use the project endpoint, deployment name, and API key from this setup.

Steps

Step 1: Create the Azure AI Foundry hub

1. In the Azure portal, search for **Azure AI Foundry**.
2. Click **+ Create** and choose **Hub**.
3. On the **Basics** tab:
   - Select the correct subscription and resource group.
   - Choose the target region, such as **USGov Virginia**.
   - Enter a hub name.
   - Create a new AI service for the hub.
4. On the **Storage** tab:
   - Select or create a storage account.
   - Set the credential store to **Azure Key Vault** and create one if needed.
   - Create a dedicated **Application Insights** instance.
   - Leave **Container registry** as **None** unless local requirements say otherwise.
5. On the **Inbound Access** tab, select **All networks**.
6. On the **Outbound Access** tab, select **Public**.
7. On the **Encryption** tab, keep the default setting unless a customer-managed key is required.
8. Review the configuration and create the hub.

Step 2: Create the project workspace

1. Open the newly created hub.
2. Launch **Azure AI Foundry** from the hub page.
3. Select **+ New project**.
4. Enter a project name, such as `llms-virginia`.
5. Confirm that the project is linked to the correct hub.
6. Create the project.

Step 3: Deploy an AI Model

1. In the project workspace, open **Models + Endpoints**.
2. Select **+ Deploy model**.

3. Choose the Azure OpenAI model that will be used later, such as `gpt-4o`. This is the **Model name** — the actual underlying model provided by Azure.
4. Enter a deployment name, such as `gpt-4o-virginia`. This is the **Deployment name** — a label you choose to identify this specific deployment. It can be anything, but something descriptive like `gpt-4o-virginia` makes it easier to manage multiple deployments.
5. Select the deployment type required for the environment.
6. Adjust the Tokens Per Minute setting if needed.
7. Create the deployment.

> **Model name vs. Deployment name**
>
> These two values are used separately in LiteLLM's `config.yaml`: the **Deployment name** goes into `model: azure/<deployment-name>`, and the **Model name** goes into `base_model`. Both are recorded in Step 4 below.

> **Deprecated models**
>
> Some models listed in the catalog cannot be deployed because they have been deprecated. If deployment fails with a `ServiceModelDeprecated` error, the selected model version is no longer available — choose a different model or a newer version. For example, `gpt-4` (version `1106-Preview`) was deprecated on 06/30/2025. Use `gpt-4o` or another active model instead.

Step 4: Locate Endpoints and Keys

1. Open the project in Azure AI Foundry. If the hub page opens instead of the project, click **Go to project** to enter the project workspace.
2. Go to the **Models + Endpoints** page.
3. Click **Get endpoint**.
4. Copy the **Azure AI model inference endpoint**.

5. Copy the **Key**.
6. Record the **Model name**.

What You Just Set Up

Azure AI Foundry is now ready for the rest of the stack. The project contains a deployed model, and the project endpoint, API key, and deployment name are available for the LiteLLM and RAG website configuration pages.

BUILD ORDER

Azure AI Search

Purpose

Create the Azure AI Search service that stores the vectorized document chunks used by the RAG workflow.

URL

Service endpoint: `https://<search-service-name>.search.azure.us`

Run Location

Azure portal.

Before You Start

- An Azure Government subscription with permission to create search resources
- A resource group in the target region
- A decision on the pricing tier needed for the project

Context

The RAG website uploads content and embeddings to Azure AI Search, and the MCP server later queries those indexes. The endpoint URL and admin API key from this page are used again in later setup steps.

Steps

Step 1: Create the Azure AI Search service

1. In the Azure portal, search for **Azure AI Search**.
2. Select **+ Create**.
3. On the **Basics** tab:
   - Choose the correct subscription and resource group.
   - Enter a service name, such as `chris-rag-testing`.
   - Select the region, such as **USGov Virginia**.
   - Choose the pricing tier. For this project, **Free** or **Basic** are sufficient in most cases. Higher tiers (S, S2, S3, etc.) are available if needed.

> **Free tier limit**
>
> Azure only allows one Free tier search service per subscription. If you already have one, creating another will fail with an error — choose **Basic** instead.

4. Review the configuration and create the service.

Step 2: Locate the endpoint and admin key

1. Open the Azure AI Search resource after deployment finishes.
2. Copy the **URL** shown on the overview page.
3. Open **Keys** in the left navigation.
4. Copy one of the **Admin keys**.

What You Just Set Up

Azure AI Search is now available for the RAG workflow. The service endpoint and admin key can be used by the RAG website to create indexes and by the MCP server to expose those indexes as tools.

BUILD ORDER

Ubuntu Virtual Machine

Purpose

Create the Ubuntu VM that hosts the private application stack:

- LiteLLM
- OpenWebUI
- RAG Website
- MCP Server

Run Location

Azure portal for VM creation, a local machine for SSH and `scp`, and the Ubuntu VM for package installation.

Before You Start

- An Azure Government subscription and resource group

Context

This VM is the host for the rest of the stack. Later pages assume the working directories live under `/home/<VM_USER_NAME>/` and that internal services are reached over WireGuard at `10.55.55.1`.

Steps

Step 1: Create the Azure VM

1. In the Azure portal, open **Virtual Machines** and select **Create**.
2. On the **Basics** tab:
   - Choose the correct subscription and resource group.
   - Enter a VM name.

- Select **Ubuntu Server 24.04**.
- Select **Standard_B2s** for the size (~$35.62/month while running). Other sizes are available if more CPU or memory is needed — click **See all sizes** to browse options by series, vCPUs, RAM, and cost.
- **(Optional)** Change the **Username** field to your desired username.
- Choose **SSH public key** for authentication.
- Allow public inbound port **SSH**.

3. On the **Networking** tab:
   - **Virtual network**: select an existing VNet or create a new one.
   - **Subnet**: leave as the default subnet.
   - **Public IP**: leave the default (a new public IP will be created).
   - **NIC network security group**: select **Basic**.
   - **Public inbound ports**: select **Allow selected ports**.
   - **Select inbound ports**: select **SSH (22)**.
   - Leave all other settings at their defaults.
4. Review the configuration and create the VM.
5. Download the private key if Azure generated one.

> **VM Pricing**
>
> Azure charges for the VM while it is in the **Running** state. To pause charges, open the VM in the portal and click **Stop** to deallocate it. Disk storage costs continue at a lower rate while deallocated. Charges resume when the VM is started again.

Step 2: Connect to the VM

On the local machine, set restrictive permissions on the private key:

```
chmod 400 /path/to/your-key.pem
```

Connect to the VM from the local machine:

```
ssh -i /path/to/your-key.pem <VM_USER_NAME>@<VM_PUBLIC_IP>
```

Step 3: Install the base packages on the VM

Run these commands on the Ubuntu VM:

```
sudo apt update
```

```
sudo apt install -y git python3 python3-pip python3-venv
curl
```

Step 4: Copy files to the VM when a later page requires it

Several later pages provide starter bundles as `.tar.gz` files. The same pattern is used each time.

From the local machine, upload the file:

```
scp -i /path/to/your-key.pem ~/Downloads/rag-website.tar.gz
<VM_USER_NAME>@<VM_PUBLIC_IP>:/home/<VM_USER_NAME>/
```

On the Ubuntu VM, extract the uploaded file:

```
tar -xzf ~/rag-website.tar.gz -C ~/
```

What You Just Set Up

The Ubuntu VM is now ready for the remaining setup pages. SSH access is in place, the base packages are installed, and later pages can add services under the VM user account.

BUILD ORDER

WireGuard

Purpose

Configure the Ubuntu VM as a WireGuard VPN gateway so the internal services can be reached over the private address `10.55.55.1`.

Run Location

Ubuntu VM unless a step explicitly says to use the Azure portal or a client device.

Before You Start

- The Ubuntu VM is created and reachable over SSH
- The VM public IP is available

Context

The WireGuard tunnel uses the subnet `10.55.55.0/24`. The VM listens as `10.55.55.1`, and the rest of the documentation assumes the private services are reached through that address instead of the public internet.

Steps

Step 1: Install WireGuard on the Ubuntu VM

```
sudo apt update
```

```
sudo apt install -y wireguard-tools
```

Step 2: Generate the server key pair

Create the `/etc/wireguard` directory and generate the keys directly into it:

```
sudo mkdir -p -m 700 /etc/wireguard
wg genkey | sudo tee /etc/wireguard/server_private.key | wg
pubkey | sudo tee /etc/wireguard/server_public.key >
/dev/null
sudo chmod 600 /etc/wireguard/server_private.key
```

You will need these values in the next steps:

```
cat /etc/wireguard/server_private.key   # paste into
wg0.conf
cat /etc/wireguard/server_public.key    # give to each VPN
client
```

Step 3: Create the server configuration

Open the configuration file:

```
sudo nano /etc/wireguard/wg0.conf
```

Paste the following template and replace the placeholders:

```
[Interface]
Address = 10.55.55.1/24
ListenPort = 51820
PrivateKey = <SERVER_PRIVATE_KEY>

PostUp = iptables -A FORWARD -i %i -j ACCEPT; iptables -A
FORWARD -o %i -j ACCEPT; iptables -t nat -A POSTROUTING -s
10.55.55.0/24 -o eth0 -j MASQUERADE
PostDown = iptables -D FORWARD -i %i -j ACCEPT; iptables -D
FORWARD -o %i -j ACCEPT; iptables -t nat -D POSTROUTING -s
10.55.55.0/24 -o eth0 -j MASQUERADE

[Peer]
PublicKey = <CLIENT_1_PUBLIC_KEY>
AllowedIPs = 10.55.55.2/32

[Peer]
PublicKey = <CLIENT_2_PUBLIC_KEY>
AllowedIPs = 10.55.55.3/32
```

Each client needs its own `[Peer]` block. On the server, `AllowedIPs` should normally be the specific client address, such as `10.55.55.2/32`.

Step 4: Enable IPv4 forwarding

```
echo "net.ipv4.ip_forward=1" | sudo tee /etc/sysctl.d/99-
wireguard.conf
```

```
sudo sysctl --system
```

Step 5: Allow WireGuard through the Azure NSG

1. Go to your virtual machine in the Azure portal.
2. In the left menu under **Settings**, click **Networking**.

3. Click **Add inbound security rule**.

Add the following two rules:

WireGuard rule

Field	Value
Source	Any
Source port ranges	*
Destination	Any
Service	Custom
Destination port ranges	51820
Protocol	UDP
Action	Allow
Priority	310
Name	AllowWireguard

SSH rule (should already exist — edit it to restrict the source)

Field	Value
Source	IP Addresses
Source IP addresses/CIDR ranges	Your company's allowed IP addresses
Destination	Any
Protocol	TCP

Step 6: Start WireGuard and enable it at boot

```
sudo systemctl enable --now wg-quick@wg0
```

```
sudo systemctl status wg-quick@wg0
```

```
sudo wg show
```

If `wg0.conf` changes later, restart the service:

```
sudo systemctl restart wg-quick@wg0
```

Step 7: Configure a client (Windows)

1. Install WireGuard

Download and install WireGuard for Windows from the official installer at wireguard.com/install.

2. Create a tunnel and generate keys

1. Open the WireGuard app.
2. Click **Add Tunnel** → **Add empty tunnel**.
3. The app generates a private key and public key automatically. The private key is filled in for you — do not share it.
4. Copy the **public key** that appears at the top of the tunnel editor.
5. Send the public key to the server administrator so they can add it as a `[Peer]` block in `/etc/wireguard/wg0.conf` on the VM.

> **Wait for confirmation**
>
> Do not activate the tunnel until the administrator confirms your public key has been added to the server and WireGuard has been restarted.

3. Configure the tunnel

In the tunnel editor, replace the contents with the following and fill in the placeholders:

```
[Interface]
PrivateKey = <CLIENT_PRIVATE_KEY>
Address = 10.55.55.2/32
DNS = 8.8.8.8

[Peer]
PublicKey = <SERVER_PUBLIC_KEY>
Endpoint = <VM_PUBLIC_IP>:51820
AllowedIPs = 10.55.55.0/24
PersistentKeepalive = 25
```

- Replace `10.55.55.2` with your assigned client VPN IP.
- `<SERVER_PUBLIC_KEY>` is the contents of `/etc/wireguard/server_public.key` on the VM.
- `<VM_PUBLIC_IP>` can be found on the **Overview** page of the VM in the Azure portal.
- `AllowedIPs = 10.55.55.0/24` is split-tunnel mode — only traffic destined for the VPN subnet routes through WireGuard.

Click **Save**.

4. Activate the tunnel

Click **Activate** in the WireGuard app. The status should change to **Active**.

Step 8: Validate the connection

From the client device, open a terminal and ping the VPN gateway:

```
ping 10.55.55.1
```

From the Ubuntu VM, confirm the client handshake:

```
sudo wg show
```

The VM should show a recent handshake for the client peer after the tunnel comes up.

Step 9: Optional full-tunnel mode

To send all client traffic through the VM, change this line in the client configuration:

```
AllowedIPs = 0.0.0.0/0
```

Keep IPv4 forwarding and the NAT rules enabled if full-tunnel mode is used.

What You Just Set Up

WireGuard now provides private access to the Ubuntu VM and the services bound to `10.55.55.1`. The rest of the application stack can stay off the public internet and still be reachable from approved client devices.

BUILD ORDER

Docker

Purpose

Install Docker Engine and Docker Compose on the Ubuntu VM so the containerized parts of the stack can run and restart automatically.

Run Location

Ubuntu VM.

Before You Start

- The Ubuntu VM is created and reachable over SSH

Context

LiteLLM and OpenWebUI run in Docker containers in this project. The Python applications use virtual environments and systemd instead.

Steps

Step 1: Install Docker repository prerequisites

```
sudo apt update
```

```
sudo apt install -y ca-certificates curl gnupg
```

```
sudo install -m 0755 -d /etc/apt/keyrings
```

Step 2: Add Docker's official apt repository

```
curl -fsSL https://download.docker.com/linux/ubuntu/gpg |
sudo gpg --dearmor -o /etc/apt/keyrings/docker.gpg
```

```
sudo chmod a+r /etc/apt/keyrings/docker.gpg
```

```
echo "deb [arch=$(dpkg --print-architecture) signed-
by=/etc/apt/keyrings/docker.gpg]
https://download.docker.com/linux/ubuntu $(lsb_release -cs)
stable" | sudo tee /etc/apt/sources.list.d/docker.list >
/dev/null
```

Step 3: Install Docker Engine and Compose

```
sudo apt update
```

```
sudo apt install -y docker-ce docker-ce-cli containerd.io
docker-buildx-plugin docker-compose-plugin
```

Step 4: Enable Docker at boot

```
sudo systemctl enable --now docker
```

Step 5: Allow the VM user to run Docker without `sudo`

```
sudo usermod -aG docker "$USER"
```

Apply the new group membership in the current shell:

```
newgrp docker
```

> **Session vs. login**
>
> `newgrp docker` applies the group change to the current shell session only. If you open a new SSH connection later, the group will already be active and `newgrp` is not needed again.

Step 6: Validate the installation

```
docker --version
```

```
docker compose version
```

```
docker ps
```

What You Just Set Up

Docker is now installed on the VM and starts automatically at boot. The VM user can run Docker commands without `sudo`, and the environment is ready for the LiteLLM and OpenWebUI setup pages.

BUILD ORDER

LiteLLM

Purpose

Run an OpenAI-compatible API on the Ubuntu VM so OpenWebUI and Cline can use an Azure OpenAI deployment through a single endpoint.

URL

- LiteLLM API: `http://10.55.55.1:4000`
- Model list endpoint: `http://10.55.55.1:4000/v1/models`

Run Location

Ubuntu VM.

Before You Start

- A model is deployed in Azure AI Foundry
- The project endpoint and API key are available from Azure AI Foundry
- Docker is installed on the VM
- WireGuard is in place so clients can reach `10.55.55.1`

Context

LiteLLM gives the rest of the stack a single OpenAI-compatible endpoint. In the example below, the Azure deployment is exposed as `gpt-4o`.

Steps

Step 1: Create the LiteLLM configuration directory

```
mkdir -p ~/lite-llm
```

```
cd ~/lite-llm
```

Step 2: Create `config.yaml`

```
nano ~/lite-llm/config.yaml
```

Paste the following configuration and replace the placeholders:

- `<deployment-name>` — the name you gave the deployment in Azure AI Foundry (e.g. `gpt-4o-virginia`). This is set in Step 3 of Azure AI Foundry.
- `<base-model-name>` — the underlying Azure OpenAI model name (e.g. `gpt-4o`). This is the **Model name** shown in Step 4 of Azure AI Foundry.

```
model_list:
  - model_name: gpt-4o
    litellm_params:
      model: azure/<deployment-name>
      api_base: os.environ/AZURE_API_BASE
      api_key: os.environ/AZURE_LLM_API_KEY
      api_version: 2025-01-01-preview
      base_model: <base-model-name>

litellm_settings:
  drop_params: true
  rate_limiter: local
```

Adding more deployments

If traffic later needs to be spread across multiple Azure deployments, additional entries can be added to `config.yaml` with the same `model_name`.

Step 3: Run LiteLLM in Docker

```
cd ~/lite-llm
```

Both values come from [Step 4 of Azure AI Foundry]():

- `AZURE_API_BASE` — the **Azure AI model inference endpoint**
- `AZURE_LLM_API_KEY` — the **Key**

```
docker run -d \
  --name litellm \
  --restart unless-stopped \
  -v "$(pwd)/config.yaml:/app/config.yaml:ro" \
  -e AZURE_API_BASE="https://<azure-openai-
endpoint>.openai.azure.us/" \
  -e AZURE_LLM_API_KEY="<AZURE_OPENAI_API_KEY>" \
  -p 10.55.55.1:4000:4000 \
  docker.litellm.ai/berriai/litellm:main-stable \
  --config /app/config.yaml
```

No API key required

This example does not set a LiteLLM master key. Clients on WireGuard can connect without sending an API key. This is intentional for this setup since the endpoint is only reachable over the private VPN tunnel.

Step 4: Validate the LiteLLM endpoint

```
curl -s http://10.55.55.1:4000/v1/models
```

```
docker logs -n 200 litellm
```

The model list should include the alias from `config.yaml`, such as `gpt-4o`.

Step 5: Update the configuration later if needed

After editing `~/lite-llm/config.yaml`, restart the container:

```
docker restart litellm
```

If the Docker environment variables change, recreate the container:

```
docker rm -f litellm
```

Run the `docker run` command again after removing the container.

What You Just Set Up

LiteLLM now provides a single OpenAI-compatible endpoint on the VM. OpenWebUI and Cline can connect to that endpoint instead of talking directly to Azure OpenAI.

BUILD ORDER

RAG Website

Purpose

Run the Flask application that manages Azure AI Search indexes, uploads PDFs, and scrapes websites for RAG content.

URL

`http://10.55.55.1:7000`

Run Location

Local machine for downloading and uploading the starter bundle, Ubuntu VM for installation and configuration, and a browser on a WireGuard-connected client for validation.

Before You Start

- Azure AI Search is created
- A text embedding model is deployed in Azure AI Foundry
- The Ubuntu VM is ready
- Python 3 and virtual environment support are installed on the VM

Context

The starter bundle expands into `~/rag-website`. It includes the application code, templates, configuration files, and a systemd unit file. After extracting it, edit `config/config.yaml` and `deploy/rag-website.service` to match the environment.

Steps

Step 1: Download the starter bundle

Download:

- RAG Website starter bundle

Step 2: Upload the bundle to the Ubuntu VM

Run this command on the local machine:

```
scp -i /path/to/your-key.pem ~/Downloads/rag-website.tar.gz
<VM_USER_NAME>@<VM_PUBLIC_IP>:/home/<VM_USER_NAME>/
```

Step 3: Extract the bundle on the Ubuntu VM

```
tar -xzf ~/rag-website.tar.gz -C ~/
```

Step 4: Edit the configuration files

Edit the application configuration:

```
nano ~/rag-website/config/config.yaml
```

```
search_config:
  endpoint: "https://<search-service-name>.search.azure.us"
  api_key: AZURE_SEARCH_KEY

embedding:
  endpoint: "https://<azure-openai-
endpoint>.openai.azure.us/"
  api_key: AZURE_LLM_API_KEY
  api_version: "2024-02-01"
  model: "<embedding-deployment-name>"
```

- `search_config.endpoint` — the **Service endpoint** from Azure AI Search Step 2 (`https://<search-service-name>.search.azure.us`)
- `embedding.endpoint` — the **Azure AI model inference endpoint** from Azure AI Foundry Step 4
- `embedding.model` — the **Deployment name** of your embedding model from Azure AI Foundry Step 3

Edit the systemd unit file:

```
nano ~/rag-website/deploy/rag-website.service
```

- `<VM_USER_NAME>` — the username on the Ubuntu VM (set during VM creation)
- `<AZURE_AI_SEARCH_API_KEY>` — the **Admin key** from Azure AI Search
- `<AZURE_OPENAI_API_KEY>` — the **Key** from Azure AI Foundry Step 4

Step 5: Create the Python virtual environment and install dependencies

```
cd ~/rag-website
```

```
python3 -m venv .venv
```

```
source .venv/bin/activate
```

```
pip install --upgrade pip
```

```
pip install -r requirements.txt
```

Step 6: Install the systemd unit

Copy the service file into systemd:

```
sudo cp ~/rag-website/deploy/rag-website.service
/etc/systemd/system/rag-website.service
```

Reload systemd:

```
sudo systemctl daemon-reload
```

Step 7: Start the service and enable it at boot

```
sudo systemctl enable --now rag-website
```

Step 8: Validate the website

```
sudo systemctl status rag-website
```

```
sudo journalctl -u rag-website -n 100 --no-pager
```

```
curl -I http://10.55.55.1:7000
```

Open the site from a client connected over WireGuard:

- `http://10.55.55.1:7000`

What You Just Set Up

The RAG website is now running on the VM and can create, update, and delete Azure AI Search indexes. It also provides the main interface for uploading documents and web content into the RAG pipeline.

BUILD ORDER

MCP Server

Purpose

Run the MCP service that exposes Azure AI Search indexes as tools for clients such as Cline and OpenWebUI.

URL

- MCP endpoint: `http://10.55.55.1:8000/mcp`
- MCPO OpenAPI proxy: `http://10.55.55.1:8001`
- MCPO docs: `http://10.55.55.1:8001/docs`

Run Location

Local machine for downloading and uploading the starter bundle, Ubuntu VM for installation and configuration, and a browser on a WireGuard-connected client for viewing the OpenAPI docs.

Before You Start

- Azure AI Search is created
- The Ubuntu VM is ready
- WireGuard is working
- The RAG Website has created at least one index

Context

The starter bundle expands into `~/mcp-server`. It includes the Python service, the MCPO bridge, and the two systemd units needed to run them at

boot. After extracting it, edit `config.py`, `deploy/mcp-server.service`, and `deploy/mcpo.service`.

Steps

Step 1: Download the starter bundle

Download:

- MCP Server starter bundle

Step 2: Upload the bundle to the Ubuntu VM

Run this command on the local machine:

```
scp -i /path/to/your-key.pem ~/Downloads/mcp-server.tar.gz
<VM_USER_NAME>@<VM_PUBLIC_IP>:/home/<VM_USER_NAME>/
```

Step 3: Extract the bundle on the Ubuntu VM

```
tar -xzf ~/mcp-server.tar.gz -C ~/
```

Step 4: Edit the configuration files

Edit the Python configuration:

```
nano ~/mcp-server/config.py
```

```
AZURE_SEARCH_ENDPOINT = "https://<search-service-
name>.search.azure.us"
AZURE_SEARCH_API_KEY = "<AZURE_AI_SEARCH_API_KEY>"
```

- `AZURE_SEARCH_ENDPOINT` — the **Service endpoint** from Azure AI Search Step 2
- `AZURE_SEARCH_API_KEY` — the **Admin key** from Azure AI Search Step 2

Edit the service files:

```
nano ~/mcp-server/deploy/mcp-server.service
```

```
nano ~/mcp-server/deploy/mcpo.service
```

- `<VM_USER_NAME>` — the username on the Ubuntu VM (set during VM creation)

Step 5: Create the Python virtual environment and install dependencies

```
cd ~/mcp-server
```

```
python3 -m venv .venv
```

```
source .venv/bin/activate
```

```
pip install --upgrade pip
```

```
pip install -r requirements.txt
```

Step 6: Install the systemd units

Copy the service files into systemd:

```
sudo cp ~/mcp-server/deploy/mcp-server.service
/etc/systemd/system/mcp-server.service
```

```
sudo cp ~/mcp-server/deploy/mcpo.service
/etc/systemd/system/mcpo.service
```

Reload systemd:

```
sudo systemctl daemon-reload
```

Step 7: Start the services and enable them at boot

```
sudo systemctl enable --now mcp-server
```

```
sudo systemctl enable --now mcpo
```

Step 8: Validate the services

```
sudo systemctl status mcp-server
```

```
sudo systemctl status mcpo
```

```
curl -i http://10.55.55.1:8000/mcp
```

```
curl -I http://10.55.55.1:8001/openapi.json
```

Open the docs from a client connected over WireGuard:

- `http://10.55.55.1:8001/docs`

What You Just Set Up

The MCP service now exposes Azure AI Search indexes as callable tools, and the MCPO bridge makes those tools available through an OpenAPI endpoint for OpenWebUI.

BUILD ORDER

OpenWebUI

Purpose

Run the web chat interface that connects to LiteLLM and can optionally use MCP tools during a conversation.

URL

`http://10.55.55.1:3000`

Run Location

Ubuntu VM for Docker commands and a browser on a WireGuard-connected client for the web interface.

Before You Start

- Docker is installed on the VM
- LiteLLM is running
- WireGuard is working
- MCP Server is optional but recommended if tool access is needed

---

Context

OpenWebUI stores users, chats, and settings in a Docker volume named `open-webui`. The first account created becomes the admin account. When the MCP tool server is enabled for a chat, those tools become available to the model in that conversation, but the model is not forced to use them.

Steps

Step 1: Start OpenWebUI in Docker

```
docker run -d \
  --name open-webui \
  --restart unless-stopped \
  -p 10.55.55.1:3000:8080 \
  -e WEBUI_AUTH=True \
  -e OPENAI_API_BASE_URL="http://10.55.55.1:4000/v1" \
  -e OPENAI_API_KEY="dummy" \
  -v open-webui:/app/backend/data \
  ghcr.io/open-webui/open-webui:main
```

Step 2: Create the first admin account

> **First account is the admin**
>
> The first account created on a fresh OpenWebUI instance automatically becomes the admin. All accounts created after that are regular users. Complete this step before sharing the URL with anyone else.

1. Open `http://10.55.55.1:3000` from a client connected over WireGuard.
2. Complete the first-run signup flow.
3. Use that first account as the OpenWebUI admin account.

Step 3: Create additional user accounts

After the admin account exists, additional users can be created through the admin panel:

1. Sign in as the admin.
2. Open **Admin Panel**.
3. Go to **Users**.
4. Create additional accounts as needed.

All accounts and settings persist in the `open-webui` Docker volume, so they survive container restarts and recreations.

Step 3a: Optional — run without accounts

If you want an open instance with no login requirement, stop and remove the current container:

```
docker rm -f open-webui
```

Then re-run with `WEBUI_AUTH=False`:

```
docker run -d \
  --name open-webui \
  --restart unless-stopped \
  -p 10.55.55.1:3000:8080 \
  -e WEBUI_AUTH=False \
  -e OPENAI_API_BASE_URL="http://10.55.55.1:4000/v1" \
  -e OPENAI_API_KEY="dummy" \
  -v open-webui:/app/backend/data \
  ghcr.io/open-webui/open-webui:main
```

> **No-auth mode**
>
> In no-auth mode anyone who can reach `http://10.55.55.1:3000` over WireGuard can use the instance without signing in. This is only appropriate if all WireGuard clients are trusted.

Step 4: Validate the container

```
curl -I http://10.55.55.1:3000
```

```
docker logs -n 200 open-webui
```

Step 4: Connect OpenWebUI to the MCP tools

1. Sign in as an admin.
2. Open **Admin Panel**.
3. Go to **Settings** -> **Integrations** -> **Manage Tool Servers**.
4. Add a connection with:
   - Type: `OpenAPI`
   - URL: `http://10.55.55.1:8001`
   - OpenAPI spec: `openapi.json`
   - Name: `MCP Server`

Step 5: Reset the instance later if needed

> **This deletes all data**
>
> These commands permanently remove all chats, users, and settings stored in the OpenWebUI volume. This cannot be undone.

To remove the container and all persisted OpenWebUI data:

```
docker rm -f open-webui
```

```
docker volume rm open-webui
```

Run the `docker run` command again after removing the container.

What You Just Set Up

OpenWebUI is now available on the private VM address and connected to LiteLLM. If the MCP tool server was added, chats can also call the search tools exposed by the MCP server.

BUILD ORDER

Visual Studio Code (Cline)

Purpose

Use Visual Studio Code with the Cline extension to connect to LiteLLM and the MCP server from a local development workstation.

Run Location

Local workstation.

Before You Start

- WireGuard is connected on the local machine
- LiteLLM is running
- MCP Server is running if tool access is needed

Context

Cline uses LiteLLM as an OpenAI-compatible provider and connects to the MCP server over Streamable HTTP. The model name entered in Cline must match the alias exposed by LiteLLM, such as `gpt-4o`.

Steps

Step 1: Install Visual Studio Code

1. Open `https://code.visualstudio.com/`.
2. Download the installer for the local operating system.
3. Install and launch Visual Studio Code.

Step 2: Install the Cline extension

1. Open Visual Studio Code.
2. Open the **Extensions** view with `Ctrl+Shift+X`.
3. Search for **Cline**.
4. Install the extension.

Step 3: Configure Cline to use LiteLLM

If this is your first time installing Cline, the setup wizard opens automatically:

1. On the "How will you use Cline?" screen, select **Bring my own API key** and click **Continue**.
2. On the "Configure your provider" screen, set:
   - **API Provider:** `OpenAI Compatible`
   - **Base URL:** `http://10.55.55.1:4000`
   - **OpenAI Compatible API Key:** `none` (a value is required — entering `none` avoids an error)
   - **Model ID:** the LiteLLM model alias, such as `gpt-4o`
3. Click **Continue**.

If Cline is already installed, open the settings directly:

1. Open the Cline panel.
2. Click the **Settings** icon in the top-right corner of the Cline panel.

3. Set the same fields: API Provider, Base URL, API Key (`none`), and Model ID.
4. Click **Done**.

Step 4: Connect Cline to the MCP server

1. Open **MCP Servers** in the Cline panel.

2. Open **Remote Servers**.
3. Add a server with:
   - **Server Name:** any label, such as `Azure MCP`
   - **Server URL:** `http://10.55.55.1:8000/mcp`
   - **Transport Type:** `Streamable HTTP`
4. Save the server entry.

Step 5: Confirm the connections

Verify the model is connected:

1. Open a new Cline chat.
2. Check the bottom bar of the Cline panel — the model name (e.g., `openai-compat:gpt-4o`) should appear there.
3. Type a short message and send it. If the model responds, LiteLLM is connected correctly.

Verify the MCP server is connected:

1. Open **MCP Servers** in the Cline panel.

2. Go to the **Configure** tab.
3. Find the MCP server entry (e.g., `Azure MCP`) and confirm it has a **green dot** on the right side, indicating it is connected and running.

What You Just Set Up

Visual Studio Code and Cline are now connected to the private VM stack. Cline can send model requests through LiteLLM and, when enabled, use the MCP tools backed by Azure AI Search.

REFERENCE

Key Terms

A reference glossary for technical terms and acronyms used throughout this documentation.

Core AI Concepts

AI (Artificial Intelligence)

The simulation of human intelligence by computer systems. In this documentation, AI refers specifically to large language models capable of understanding and generating text.

LLM (Large Language Model)

A type of AI model trained on large amounts of text data that can understand and generate human language. Examples include GPT-4o. LLMs power the chat and reasoning capabilities in this stack.

RAG (Retrieval-Augmented Generation)

A technique that enhances an LLM's responses by first retrieving relevant documents from a knowledge base and providing them as context. Instead of relying solely on the model's training data, RAG allows the model to answer questions using your own documents.

MCP (Model Context Protocol)

An open protocol that standardizes how AI models connect to external tools and data sources. MCP servers expose capabilities (like web search or file access) that an AI model can call during a conversation.

MCPO (MCP OpenAPI Proxy)

A proxy that wraps MCP servers and exposes them as an OpenAPI-compatible HTTP endpoint. This allows tools like OpenWebUI to communicate with MCP servers over a standard REST interface.

Embedding / Vector Embedding

A numerical representation of text as a list of numbers (a vector). Embeddings capture the semantic meaning of text so that similar content produces similar vectors. Azure AI Search stores these embeddings to enable similarity-based document retrieval.

Chunking

The process of splitting large documents into smaller segments before generating embeddings. Chunking ensures that the pieces of text fed into the model are small enough to be processed effectively and retrieved accurately.

Token / TPM (Tokens Per Minute)

A token is the basic unit of text that a language model processes — roughly equivalent to a word or word fragment. TPM (Tokens Per Minute) is a rate limit that controls how many tokens a model deployment can handle per minute.

Inference

The process of running an AI model to generate a response. When you send a message to an LLM, the model performs inference to produce an output.

Azure & Cloud Services

Azure AI Foundry

Microsoft's cloud platform for hosting and deploying AI models. In this stack, Azure AI Foundry is used to deploy the LLM (GPT-4o) and the embedding model, providing API endpoints that the rest of the system calls.

Azure AI Search

A cloud search service from Microsoft that stores vector embeddings and supports semantic similarity search. It acts as the knowledge base in the RAG pipeline — documents are indexed here and retrieved at query time.

Azure Government

A version of Microsoft Azure designed for U.S. government agencies and contractors, hosted in separate datacenters with additional compliance controls. This documentation targets Azure Government deployments.

API (Application Programming Interface)

A defined interface that allows one software system to communicate with another. In this stack, the LLM and search services are accessed via HTTP-based APIs.

API Key

A secret string used to authenticate requests to an API. Components in this stack (LiteLLM, RAG website, MCP server) use API keys to prove they are authorized to call Azure AI Foundry and Azure AI Search.

Resource Group

An Azure organizational container that holds related resources (VMs, search services, AI deployments) for a project. Resources in the same group share a lifecycle and can be managed together.

VM (Virtual Machine)

A software-based computer that runs on physical hardware in a cloud datacenter. In this stack, an Ubuntu VM hosted on Azure runs Docker, LiteLLM, the RAG website, the MCP server, and OpenWebUI.

Deployment

In the Azure AI Foundry context, a deployment is a named instance of a model (e.g., `gpt-4o`) that has been provisioned with a specific capacity and assigned an API endpoint.

Networking & Security

VPN (Virtual Private Network)

An encrypted network tunnel that connects two networks or devices securely over the internet. A VPN is used in this stack so that clients outside the Azure VM can access internal services privately.

WireGuard

A modern, lightweight VPN protocol used in this stack to create a secure tunnel between client machines and the Azure VM. WireGuard is known for its simplicity and high performance.

VNet (Virtual Network)

Azure's private network environment where resources like VMs are placed. The VM in this stack resides in a VNet, which controls what traffic can reach it.

NSG (Network Security Group)

An Azure firewall that filters inbound and outbound traffic to resources in a VNet. NSG rules are configured to allow only the necessary ports (e.g., SSH, WireGuard) to reach the VM.

NAT (Network Address Translation)

A technique that maps traffic from one IP address range to another. In the WireGuard setup, NAT allows VPN clients to route traffic through the VM to reach Azure services.

NIC (Network Interface Card)

The virtual network adapter attached to the Azure VM. The NIC connects the VM to the VNet and is associated with the public IP address and NSG rules.

Split Tunnel / Full Tunnel

Two VPN routing modes. In **split tunnel** mode, only traffic destined for the VPN network is routed through the VPN — other traffic goes directly to the internet. In **full tunnel** mode, all traffic is routed through the VPN.

DNS (Domain Name System)

The system that translates human-readable domain names (e.g., `example.com`) into IP addresses. DNS settings in WireGuard determine how VPN clients resolve domain names.

SSH (Secure Shell)

A cryptographic network protocol for securely connecting to a remote machine's command line. SSH is used to access and configure the Azure VM during setup.

Infrastructure & Application Stack

Docker / Container

Docker is a platform for packaging applications and their dependencies into self-contained units called **containers**. Containers run consistently across different environments. In this stack, Docker runs LiteLLM and OpenWebUI.

Docker Compose

A tool for defining and running multi-container Docker applications using a single `docker-compose.yml` configuration file. Used in this stack to manage the LiteLLM and OpenWebUI containers together.

systemd

The service manager for Linux systems. After setup, the RAG website and MCP server run as systemd services so they start automatically on boot and can be managed with `systemctl` commands.

Flask

A lightweight Python web framework. The RAG website is built with Flask and serves the document upload and query interface on port 7000.

OpenAPI

A standard specification format for describing REST APIs. The MCP server's tools are exposed via an OpenAPI-compatible endpoint so that clients like OpenWebUI can discover and call them.

LiteLLM

An open-source proxy that provides a unified OpenAI-compatible API endpoint in front of various LLM backends. In this stack, LiteLLM connects to Azure AI Foundry and runs on port 4000, allowing other services to call the LLM through a single consistent interface.

OpenWebUI

A web-based chat interface that connects to LLMs and MCP tools. It runs in a Docker container on port 3000 and provides the primary user-facing chat experience in this stack.

Cline

A VS Code extension that adds an AI coding assistant to the editor. Cline connects to LiteLLM (for the LLM) and the MCP server (for tools), enabling AI-assisted development directly inside VS Code.

REFERENCE

Web Browser

Purpose

Use a browser to access the web UIs in this stack over WireGuard.

URLs

- **OpenWebUI:** `http://10.55.55.1:3000`
- **RAG Website:** `http://10.55.55.1:7000`
- **MCP Docs:** `http://10.55.55.1:8001/docs`

REFERENCE

Azure API

Purpose

It’s the set of HTTPS web endpoints your app stack calls inside **Microsoft Azure Government** to use cloud services like **Azure AI Search** (querying indexes) and **Azure AI Foundry / Azure OpenAI** (sending prompts to a deployed model like GPT-4o). These calls are made over the internet using standard REST APIs, authenticated with an API key or Azure identity, and return JSON responses your services (LiteLLM, MCP Server, RAG Website, OpenWebUI) use to produce answers.